Seven Recommenders on MovieLens-1M

Rating prediction, Top-10 recommendation, and a fairness audit

Daize Dong Jiawei Wu
dd1376, jw2070

CS 550 Massive Data Mining and Learning
Rutgers University

Spring 2026

What we built

- Seven recommender algorithms, one codebase, one split.
- Two tasks per the course spec: rating prediction and Top-10.
- Everything written from NumPy/PyTorch primitives; no Surprise, no implicit, no lightfm.

The interesting part is not the numbers on any one model. It is what the comparison tells you about which choices matter.

Data

MovieLens-1M: $1{,}000{,}209$ integer ratings, $6{,}040$ users, $3{,}706$ movies, 20-core on users.

- Drop items with < 5 ratings to stabilise similarity estimation. Left with N_u=6040, N_i=3416.
- Per-user hold-out, seed 42: ≈ 800k train / 200k test. 95.16% sparse.

The seven models

Group	Model	One-line
Baseline	GlobalMean	constant μ
Baseline	BiasOnly	$\mu + b_u + b_i$ by alternating ridge
Memory CF	ItemKNN	adjusted cosine, shrinkage, top-50
Memory CF	UserKNN	centred cosine, shrinkage, top-50
Factor	MF	biased SVD, $d{=}64$, Adam, MSE
Neural	NeuMF	GMF $\oplus$ MLP, He 2017
Ranking	BPR-MF	pairwise logistic, Rendle 2009
Graph	LightGCN	3-layer symmetric propagation, He 2020

Methods: memory-based CF and NeuMF

ItemKNN / UserKNN. Shrinkage-adjusted cosine on mean-centred ratings. For items:

$$s_{ij} = \frac{\sum_{u \in \mathcal{U}_{ij}} \tilde{r}_{ui}\, \tilde{r}_{uj}}{\|\tilde{r}_{\cdot i}\|\, \|\tilde{r}_{\cdot j}\|} \cdot \frac{|\mathcal{U}_{ij}|}{|\mathcal{U}_{ij}| + \alpha}, \qquad \alpha = 100,\ k = 50.$$

Predict $\hat{r}_{ui} = \bar{r}_u + \big(\sum_{j \in N_k(i) \cap \mathcal{I}_u} s_{ij}\, \tilde{r}_{uj}\big) / \sum_j |s_{ij}|$. UserKNN is the symmetric version on the user side.

NeuMF (He et al., 2017). Two branches concatenated, then a linear head:

- GMF: $p_u^g \odot q_i^g$
- MLP: $[p_u^m \| q_i^m]$ through hidden sizes $(64, 32, 16)$, ReLU + Dropout(0.1)

Explicit rating target $\Rightarrow$ trained with MSE (not the original BCE).

Methods: MF and BPR-MF share the scoring function

Same scoring function $\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$. Only the loss differs.

Biased MF (point-wise MSE):

$$\mathcal{L}_{\text{MF}} \;=\; \sum_{(u,i)\in\mathcal{D}_{\text{train}}} \big(r_{ui} - \hat{r}_{ui}\big)^2 + \lambda \left\|\Theta\right\|^2.$$

$d = 64$, Adam lr 10^{-2}, batch 8192, 20 epochs, $\lambda = 10^{-4}$.

BPR-MF (pair-wise logistic, Rendle 2009). Sample triples (u, i, j) with i observed, j unobserved:

$$\mathcal{L}_{\text{BPR}} \;=\; -\sum_{(u,i,j)} \log \sigma\big(\hat{x}_{ui} - \hat{x}_{uj}\big) + \lambda \left\|\Theta\right\|^2, \qquad \hat{x}_{ui} = p_u^\top q_i + b_i.$$

Same d, Adam lr 10^{-2}, batch 4096, 20 epochs, $\lambda = 10^{-5}$.

Methods: LightGCN

Strip feature transforms and non-linearities from the GCN. Keep only weighted-average propagation on the user-item bipartite graph (He et al., 2020).

Let $\hat{\mathbf{A}} = \mathbf{D}^{-1/2}\mathbf{A}\,\mathbf{D}^{-1/2}$ be the symmetrically normalised adjacency. Each layer:

$$\mathbf{E}^{(\ell)} = \hat{\mathbf{A}}\,\mathbf{E}^{(\ell-1)}.$$

Final embedding averages across all layers:

$$\mathbf{E}^{*} = \frac{1}{L+1}\sum_{\ell=0}^{L}\mathbf{E}^{(\ell)}.$$

Trained with the same BPR loss as BPR-MF. We use $L = 3$, $d = 64$. Two budget configurations: short run at batch $16{,}384$ for wall-clock comparison, and a matched run at batch $4{,}096$ with the same gradient-step count as BPR-MF.

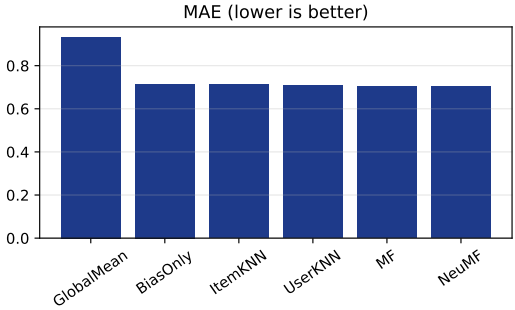
Evaluation

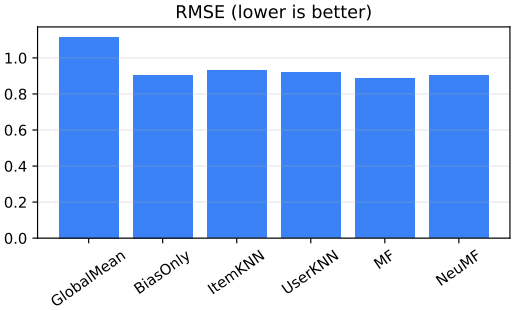
Rating. MAE and RMSE on held-out (u, i) pairs, $\hat{r}$ clipped to $[1, 5]$.

Top-10. For each user: score all items, mask items in training, take top-10, compute P / R / F_1 / NDCG against held-out positives. Average over users with ≥ 1 test item.

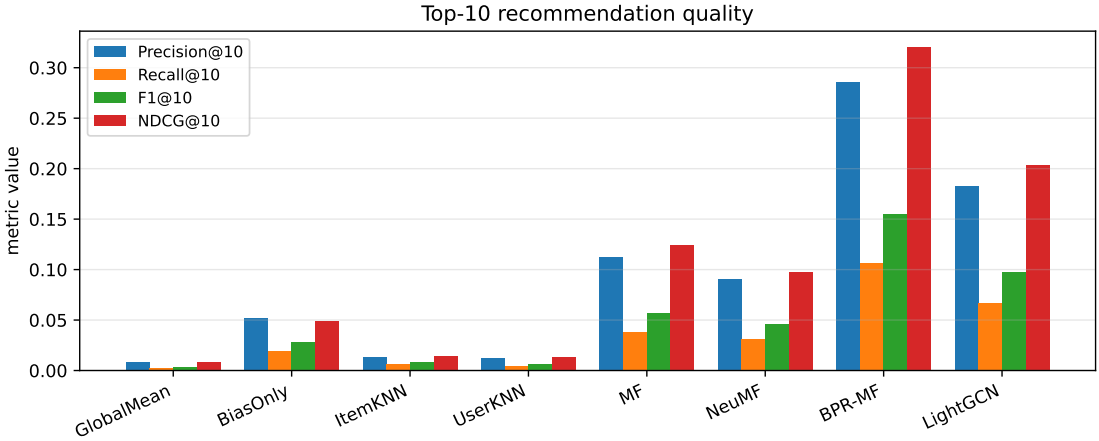
BPR-MF and LightGCN produce uncalibrated scores; reporting their MAE/RMSE would be misleading, so those cells are N/A.

Rating prediction





Top-10 recommendation



Finding 1: the loss matters more than the architecture

BPR-MF and MF use the *same* scoring function $p_u^\top q_i + b_i$. Only the loss differs.

Model	NDCG@10	P@10
MF (MSE)	0.125	0.112
BPR-MF (BPR)	**0.320**	**0.286**

MSE spends capacity on the 2s and 3s. BPR spends every gradient step on ordering positives above unobserved items, which is what the ranking metrics reward.

Why kNN is good on rating but bad on ranking

UserKNN and ItemKNN match MF on MAE, but trail it by $10\times$ on ranking.

For unseen items the neighbourhood estimate collapses back to $\bar{r}_u$: the neighbours that drive the prediction are mostly items the user has already rated, and those are masked out for Top-N. The remaining scores stop discriminating.

Practical fix: use kNN as a candidate generator, then rerank with a ranking model.

Finding 2: LightGCN does not beat BPR-MF here

First run (15 epochs, batch 16k) was much fewer gradient steps than BPR-MF. Easy objection: undertrained. So we reran with matched steps.

Config	Epochs	Batch	Grad. steps	NDCG@10
LightGCN, short	15	16,384	∼720	0.204
LightGCN, matched	20	4,096	∼3,900	0.204
BPR-MF	20	4,096	∼3,900	**0.320**

The matched run gives the same numbers at $46\times$ the wall-clock. The gap is not a compute artefact.

Most likely cause, by the paper's own Fig. 4: $L{=}3$ is too many layers for ML-1M. The paper's ML-1M optimum is $L{=}2$. We did not sweep L.

Fairness audit

(Optional Task: Fairness and Unbiases)

Bonus task from the project description. If a model funnels every recommendation to the most popular 10 movies, it will look fine on NDCG and terrible on the tail. Worth checking.

Model	Coverage@10	Gini	Long-tail @10
GlobalMean	0.103	0.987	0.873
BiasOnly	0.095	0.995	0.260
ItemKNN	0.821	0.690	0.557
UserKNN	0.757	0.749	0.870
MF	0.052	0.994	0.015
NeuMF	0.480	0.876	0.257
BPR-MF	0.204	0.975	0.005
LightGCN	**0.022**	0.995	**0.000**

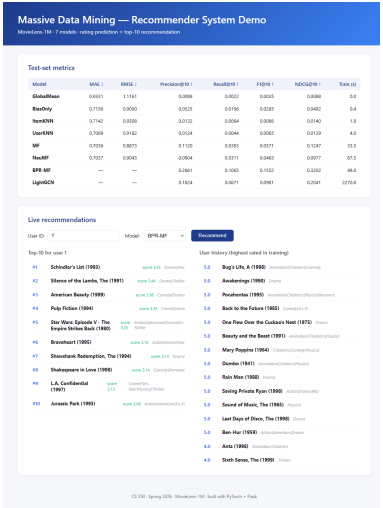
LightGCN's top-10 never leaves the popularity head and touches only 2% of the catalogue. The two NDCG winners are the two worst on the fairness numbers.

Web demo

Flask + vanilla JS. Pick a user ID, pick a model, get a top-10 list next to that user's top-rated training history.

All seven pickled models load at startup; per-user inference is a few milliseconds.

`python -m demo.app` → `localhost:5000`



What we did not do

- Cold start. Every model is transductive; extending to side information is a separate project.
- Temporal dynamics. Timestamps are in the data but the models do not read them.
- Layer sweep for LightGCN ($L{=}2$ probably fixes it).
- Multiple seeds / bootstrap CIs. Single seed is a rubric weakness.
- LLM-based recommenders (P5, SASRec). Adjacent to the spec, not included.

Bottom line

- Loss function moved the ranking metrics more than any architectural change.
- LightGCN lost on ML-1M under a matched budget; $L{=}3$ is the suspect.
- The NDCG winners are also the worst popularity-concentrators. "Use BPR in production" is the right first move, not the last.

Questions?